

Section 9

Task 10

FLIPFLOPS, REGISTERS AND RESET

23-07-2026

PLAYGROUND LINK: <https://www.edaplayground.com/x/ZqbB>

GIT LINK: https://github.com/Nooorrr66/MEDS_2026/tree/main/module_4/8_Multiplexers_Shifters

CODE:

Test Bench

```
module tb;
logic clk;
logic rst_n;
logic up_down;
logic [3:0] count;
up_down_counter dut(
    .clk(clk),
    .rst_n(rst_n),
    .up_down(up_down),
    .count(count)
);
always #5 clk = ~clk;

initial
begin
    $dumpfile("up_down_counter.vcd");
    $dumpvars(0, tb);
    clk = 0;
    rst_n = 1;
    up_down = 1;
    repeat(4) @(posedge clk);
    $display("Count before reset = %0d", count);

    #2;
    rst_n = 0;

    #1;
    $display("Count immediately after reset = %0d",
count);

    #7;
    rst_n = 1;
    up_down = 1;
    repeat(18)
    begin
        @(posedge clk);
        $display("UP Count = %0d", count);
    end
end
```

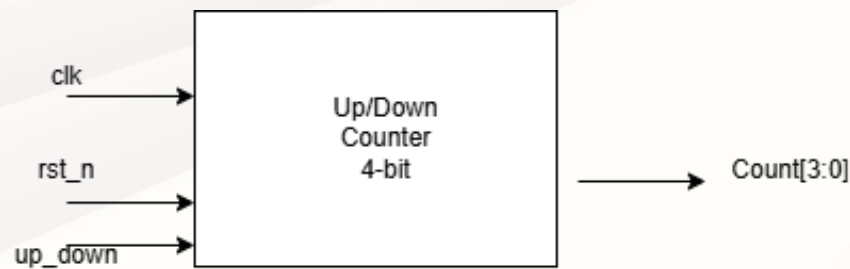
```
up_down = 0;
repeat(18)
begin
    @(posedge clk);
    $display("DOWN Count = %0d", count);
end
$finish;
end
endmodule
```

Design:

```
module up_down_counter (
    input logic clk,
    input logic rst_n,
    input logic up_down,
    output logic [3:0] count
);

always_ff @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
        count <= 4'b0000;
    else if (up_down)
        count <= count + 1;
    else
        count <= count - 1;
end
endmodule
```

SCHEMATIC:

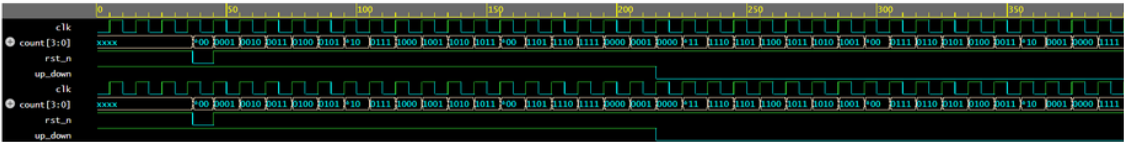


TERMINAL:

```
Log Share
Count before reset = x
Count immediately after reset = 0
UP Count = 0
UP Count = 1
UP Count = 2
UP Count = 3
UP Count = 4
UP Count = 5
UP Count = 6
UP Count = 7
UP Count = 8
UP Count = 9
UP Count = 10
UP Count = 11
UP Count = 12
UP Count = 13
UP Count = 14
UP Count = 15
UP Count = 0
UP Count = 1
```

```
Log Share
UP Count = 9
UP Count = 10
UP Count = 11
UP Count = 12
UP Count = 13
UP Count = 14
UP Count = 15
UP Count = 0
UP Count = 1
DOWN Count = 0
DOWN Count = 15
DOWN Count = 14
DOWN Count = 13
DOWN Count = 12
DOWN Count = 11
DOWN Count = 10
DOWN Count = 9
DOWN Count = 8
DOWN Count = 7
DOWN Count = 6
DOWN Count = 5
DOWN Count = 4
DOWN Count = 3
DOWN Count = 2
DOWN Count = 1
DOWN Count = 0
DOWN Count = 15
$finish called from file "testbench.sv", line 46.
$finish at simulation time 395
```

WAVEFORM:



EXPLANATION:

The design implements a 4-bit up/down counter using an `always_ff` block with an active-low asynchronous reset. The sensitivity list includes both `posedge clk` and `negedge rst_n`, allowing the counter to reset immediately whenever `rst_n` is asserted low. When reset is inactive, the counter increments by one on each rising clock edge if `up_down` is high, and decrements by one if `up_down` is low. Since the counter is 4 bits wide, it naturally wraps from 15 to 0 while counting up and from 0 to 15 while counting down.

The testbench generates a periodic clock and applies different input stimuli to verify the counter's functionality. It first allows the counter to count for a few clock cycles, then asserts the asynchronous reset between clock edges to confirm that the counter clears immediately. After releasing the reset, the testbench verifies the counting-up sequence, followed by the counting-down sequence, while printing the counter value at each clock edge. This ensures that all required behaviors, including wraparound, are tested.

The terminal output displays the counter value throughout the simulation. It shows the counter incrementing before reset, immediately clearing to 0 when the asynchronous reset is asserted, then resuming normal operation after reset is released. The output also confirms correct incrementing, decrementing, and wraparound between 15 and 0, demonstrating that the design behaves as expected.

The waveform confirms the sequential operation of the counter. The count output changes only on the rising edge of the clock during normal operation, while the asynchronous reset forces the counter to 0000 immediately when `rst_n` goes low, regardless of the clock. After the reset is deasserted, counting resumes on subsequent clock edges. The waveform also clearly shows both the count-up and count-down sequences, including the expected wraparound from 1111 to 0000 and from 0000 to 1111.

Task 11

PLAYGROUND LINK:

<https://www.edaplayground.com/x/hPVa>

CODE:

Test Bench

```
`timescale 1ns/1ps
module tb;
  logic clk;
  logic rst_n_async;
  logic rst_n_sync;
  // Instantiate DUT
  reset_synchronizer dut (
    .clk(clk),
    .rst_n_async(rst_n_async),
    .rst_n_sync(rst_n_sync)
  );
  // Clock Generation (10 ns period)
  always #5 clk = ~clk;
  // Waveform Dump
  initial begin
    $dumpfile("reset_sync.vcd");
    $dumpvars(0, tb);
  end
  // Monitor Signals
  initial begin
    $display("-----");
    $display(" Time\tclk\ttrst_n_async\ttrst_n_sync");
    $display("-----");
    $monitor("%4t\t%b\t%b\t%b\t%b",
      $time, clk, rst_n_async, rst_n_sync);
  end
  // Stimulus
  initial begin
    // Initial values
    clk = 0;
    rst_n_async = 1;
    // Test 1 : Assert reset between clock edges
    #7;
    $display("\nTEST 1: Assert asynchronous reset");
    rst_n_async = 0;
    #13;
    $display("Release reset between clock edges");
    rst_n_async = 1;
    // Wait for synchronization
    repeat(4) @(posedge clk);
    // Test 2 : Assert reset exactly at clock edge
    $display("\nTEST 2: Assert reset at clock edge");
    @(posedge clk);
    rst_n_async = 0;
    #8;
    $display("Release reset close to next clock edge");
    rst_n_async = 1;
    repeat(5) @(posedge clk);
    // Test 3 : Quick reset pulse
    $display("\nTEST 3: Short asynchronous reset pulse");
    #3;
    rst_n_async = 0;
    #6;
    rst_n_async = 1;
    repeat(5) @(posedge clk);
    $display("\nSimulation Complete.");
    $finish;
  end
end
endmodule
```

Design:

```
module reset_synchronizer(
  input  logic clk,
  input  logic rst_n_async,
  output logic rst_n_sync
);
  logic ff1, ff2;

  always_ff @(posedge clk or negedge rst_n_async)
  begin
    if (!rst_n_async)
      begin
        ff1 <= 1'b0;
        ff2 <= 1'b0;
      end
    else
      begin
        ff1 <= 1'b1;
        ff2 <= ff1;
      end
    end
  end
  assign rst_n_sync = ff2;
endmodule
```

TERMINAL:

Log

Share

Time	clk	rst_n_async	rst_n_sync
0	0	1	x
5000	1	1	x

TEST 1: Assert asynchronous reset

7000	1	0	0
10000	0	0	0
15000	1	0	0

Release reset between clock edges

20000	0	1	0
25000	1	1	0
30000	0	1	0
35000	1	1	1
40000	0	1	1
45000	1	1	1
50000	0	1	1

TEST 2: Assert reset at clock edge

55000	1	1	1
60000	0	1	1
65000	1	0	0
70000	0	0	0

Release reset close to next clock edge

73000	0	1	0
75000	1	1	0
80000	0	1	0
85000	1	1	1
90000	0	1	1
95000	1	1	1
100000	0	1	1
105000	1	1	1
110000	0	1	1

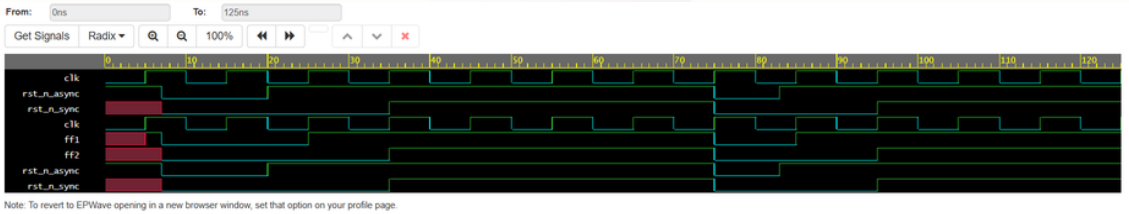
110000	0	1	1
--------	---	---	---

TEST 3: Short asynchronous reset pulse

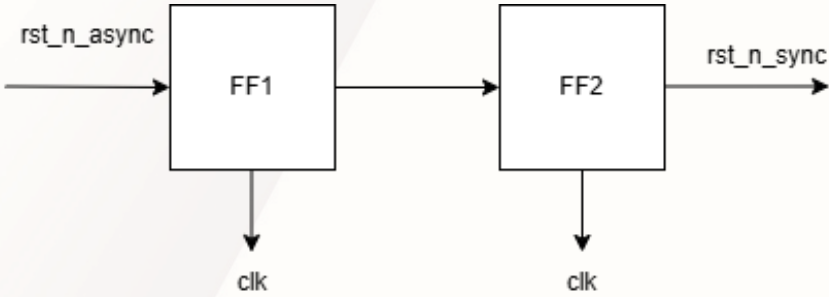
115000	1	1	1
118000	1	0	0
120000	0	0	0
124000	0	1	0
125000	1	1	0
130000	0	1	0
135000	1	1	1
140000	0	1	1
145000	1	1	1
150000	0	1	1
155000	1	1	1
160000	0	1	1

Simulation Complete.
\$finish called from file "testbench.sv", line 59.
\$finish at simulation time 165000
VCS Simulation Report

WAVEFORM:



SCHEMATIC



EXPLANATION:

The design implements a two-flip-flop reset synchronizer that converts an asynchronous active-low reset into a synchronized reset signal. Both flip-flops use an asynchronous active-low reset, allowing the reset to be asserted immediately regardless of the clock. When `rst_n_async` is deasserted, the first flip-flop captures a logic high on the next rising clock edge, and the second flip-flop captures it on the following clock edge. As a result, the synchronized reset (`rst_n_sync`) is released only after two clock cycles, ensuring that reset deassertion occurs safely on a clock edge and reducing the risk of metastability.

The testbench generates a periodic clock and applies several reset scenarios to verify the synchronizer's behavior. It first asserts the asynchronous reset between clock edges, then releases it to observe the synchronized deassertion. Additional tests assert the reset at or near a clock edge and apply a short reset pulse to ensure consistent operation under different timing conditions. A waveform dump and signal monitor are included to observe the behavior of the reset signals throughout the simulation.

The terminal output displays the values of `clk`, `rst_n_async`, and `rst_n_sync` during the simulation. It confirms that whenever the asynchronous reset is asserted, the synchronized reset immediately becomes active. After the asynchronous reset is released, the output remains in the reset state until two rising clock edges have occurred, demonstrating the intended synchronization delay. The printed values verify that the synchronizer behaves correctly under all test cases.

The waveform illustrates the operation of the reset synchronizer. When `rst_n_async` is asserted low, both internal flip-flops (`ff1` and `ff2`) and the synchronized reset (`rst_n_sync`) are cleared immediately without waiting for a clock edge. When the asynchronous reset is deasserted, `ff1` changes state on the first rising edge of the clock, while `ff2` changes on the following rising edge. Consequently, `rst_n_sync` is released only after two clock cycles and always transitions on a clock edge, preventing recovery-time violations and ensuring a stable, synchronized reset signal for the rest of the circuit.